

SC2001

ALGORITHM DESIGN AND ANALYSIS

Project 1: Integration of Merge Sort & Insertion Sort

In merge sort, when the sizes of subarrays are small, the overhead of many recursive calls makes the algorithm inefficient. Therefore, in real use, we often combine merge sort with Insertion Sort to come up with a hybrid sorting algorithm for better efficiency. The idea is to set a small integer **S** as a threshold for the size of subarrays. Once the size of a subarray in a recursive call of merge sort is less than or equal to **S**, the algorithm will switch to Insertion Sort, which is efficient for small-sized input.

- (a) **Algorithm implementation:** Implement the above hybrid algorithm.
- (b) **Generate input data:** Generate arrays of increasing sizes, in a range from 1,000 to 10 million. For each of the sizes, generate a random dataset of integers in the range of $[1, \dots, x]$, where x is the largest number you allow for your datasets.
- (c) **Analyze time complexity:** Run your program of the hybrid algorithm on the datasets generated in Step (b). Record the number of key comparisons performed in each case.
 - i. With the value of **S** fixed, plot the number of key comparisons over different sizes of the input list n . Compare your empirical results with your theoretical analysis of the time complexity.
 - ii. With the input size n fixed, plot the number of key comparisons over different values of **S**. Compare your empirical results with your theoretical analysis of the time complexity.
 - iii. Using different sizes of input datasets, study how to determine an optimal value of **S** for the best performance of this hybrid algorithm.
- (d) **Compare with original Merge Sort:** Implement the original version of merge sort (as learnt in lecture). Compare its performance against the above hybrid algorithm in terms of the number of key comparisons and CPU times on the dataset with 10 million integers. You can use the optimal value of **S** obtained in (c) for this task.